CSL 100/ MAL 100

Tutorial 8

Recursive function and Array and pointer

1) Find Maximum Subarray Sum:

Given an integer array arr[] of size n, your task is to write a C program to find the subarray (contiguous part of the array) which has the maximum sum and return that sum.

Input: Input: arr[] = {2, 3, -8, 7, -1, 2, 3}

Output: 11

Explanation: The subarray {7, -1, 2, 3} has the largest sum 11.

Input: arr[] = {-2, -4}

Output: –2

Explanation: The subarray {-2} has the largest sum -2.

Input: arr[] = {5, 4, 1, 7, 8}

Output: 25

Explanation: The subarray {5, 4, 1, 7, 8} has the largest sum 25.

```
#include <stdio.h>
int maxSubarraySum(int arr[], int size) {
    int maxSum = arr[0];
    for (int i = 0; i < size; i++) {
        int currSum = 0;
        for (int j = i; j < size; j++) {
            currSum = currSum + arr[j];
            if (currSum > maxSum) {
                maxSum = currSum;
            }
        }
    }
    return maxSum;
}

int main() {
    int arr[] = {2, 3, -8, 7, -1, 2, 3};
    int size = sizeof(arr) / sizeof(arr[0]);
    printf("%d", maxSubarraySum(arr, size));
    return 0;
}
```

The idea is to run two nested loops to iterate over all possible subarrays and find the maximum sum. The outer loop will mark the starting point of a subarray and inner loop will mark the ending point of the subarray.

2) Find Subarray with Given Sum

You are given an integer array `arr[]` of size `n` (using **1-based indexing**) and an integer `sum`. Your task is to write a C program that finds the **first subarray** (contiguous part of the array) whose sum equals the given `sum`. If multiple such subarrays exist, return the **indexes of the first subarray** (moving from left to right). If no such subarray exists, return `-1`.

Input: Input: arr[] = { 15, 2, 4, 8, 9, 5, 10, 23}, sum = 23

Output: 2 5

Explanation: Sum of elements between indices 2 and 5 is 2 + 4 + 8 + 9 = 23

Input: arr[] = {1, 4, 0, 0, 3, 10, 5}, sum = 7

Output: 2 5

Explanation: Sum of elements between indices 2 and 5 is 4 + 0 + 0 + 3 = 7

Input: arr[] = {1, 4}, sum = 0

Output: -1

Explanation: There is no subarray with 0 sum

```c
#include <stdio.h>

void findSubarrayWithSum(int arr[], int n, int targetSum) {
    int start = 0, currentSum = 0;

    for (int end = 0; end < n; end++) {
        currentSum += arr[end];
        while (currentSum > targetSum && start <= end) {
            currentSum -= arr[start];
            start++;
        }
        if (currentSum == targetSum) {
            printf("%d %d\n", start + 1, end + 1);
            return;
        }
    }
    printf("-1\n");
}
```

```c
int main() {
    int n, targetSum;
    printf("Enter the number of elements: ");
    scanf("%d", &n);

    int arr[n];
    printf("Enter the array elements: ");
    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }
    printf("Enter the target sum: ");
    scanf("%d", &targetSum);
    findSubarrayWithSum(arr, n, targetSum);
    return 0;
}
```

For each starting element, the inner loop iterates through subsequent elements and adding each element to the cumulative sum until the given sum is found or the end of the array is reached. If at any point the cumulative sum equals the given sum, then return starting and ending indices (1-based). If no such sub-array is found after all iterations, then return -1.

3) Coin Change Problem

You are given an array coins[] of size N representing different coin denominations and an integer sum. Your task is to write a C program that counts all possible combinations of coins to make the given value sum. You can use an infinite supply of each coin denomination.

Input: Input: sum = 4, coins[] = {1,2,3}

Output: 4

Explanation: there are four solutions: {1, 1, 1, 1}, {1, 1, 2}, {2, 2} and {1, 3}

Input: sum = 10, coins[] = {2, 5, 3, 6}

Output: 5

Explanation: There are five solutions:

{2,2,2,2,2}, {2,2,3,3}, {2,2,6}, {2,3,5} and {5,5}

Input: sum = 10, coins[] = {10}

Output: 1

Explanation: The only is to pick 1 coin of value 10.

Input: sum = 5, coins[] = {4}

Output: 0

Explanation: We cannot make sum 5 with the given coins

```
#include <stdio.h>
int count(int coins[], int n, int sum)
{
    if (sum == 0)
        return 1;
    if (sum < 0)
        return 0;
    if (n <= 0)
        return 0;
    return count(coins, n - 1, sum)
          + count(coins, n, sum - coins[n - 1]);
}
int main()
{
    int i, j;
    int coins[] = { 1, 2, 3 };
    int n = sizeof(coins) / sizeof(coins[0]);
    printf("%d ", count(coins, n, 5));
    getchar();
    return 0;
}
```

For each coin, there are 2 options.

Include the current coin: Subtract the current coin's denomination from the target sum and call the count function recursively with the updated sum and the same set of coins i.e., count(coins, n, sum – coins[n-1])

Exclude the current coin: Call the count function recursively with the same sum and the remaining coins. i.e., count(coins, n-1,sum).

Question

K’th Smallest Element in Unsorted Array

- **Problem Statement:** Given an array arr[] of N distinct elements and a number K, where K is smaller than the size of the array. Find the K’th smallest element in the given array.

example Input: arr[] = {7, 10, 4, 3, 20, 15}, K = 3

Output: 7

Input: arr[] = {7, 10, 4, 3, 20, 15}, K = 4

Output: 10